

Advanced JavaScript Mastery

Execution Contexts, Memory Models, and Language Semantics

Part 1: Scope, Bindings, and the Temporal Dead Zone

What is the difference between `let`, `const`, and `var`?

`var` is function-scoped (or globally scoped), hoisted, and initialized with `undefined`. `let` and `const` are block-scoped, hoisted but *not* initialized, placing them in the Temporal Dead Zone (TDZ) until evaluation. `const` additionally prevents reassignment.

What is the difference between declaration, initialization, and assignment?

Declaration registers a variable name in the lexical environment. **Initialization** allocates memory and assigns an initial state (like `undefined` for `var`). **Assignment** binds a specific data value to the allocated variable at runtime.

What exactly is the Temporal Dead Zone (TDZ), and why does it exist?

The TDZ is the period between the creation of a variable's lexical binding and its actual initialization. Accessing a `let` or `const` variable inside its TDZ throws a `ReferenceError`. It was designed to catch bugs early by preventing the use of variables before their values are evaluated, ensuring predictable state.

Why can `typeof` throw a `ReferenceError` for a `let` or `const` variable?

Historically, `typeof undeclaredVar` safely returned `"undefined"`. However, if a variable is declared with `let` or `const` later in the same block, accessing it beforehand traps it in the TDZ. The engine strictly enforces TDZ rules over the historical safety of `typeof`.

What is variable shadowing?

Shadowing occurs when a variable declared in an inner lexical scope shares the exact same identifier as a variable in an outer scope. The inner binding takes precedence, blocking direct access to the outer variable within that block.

What does `globalThis` solve?

Historically, accessing the global object required different syntax across environments (`window` in browsers, `global` in Node.js, `self` in Web Workers). `globalThis` provides a standardized, universal reference to the top-level execution environment regardless of the host runtime.

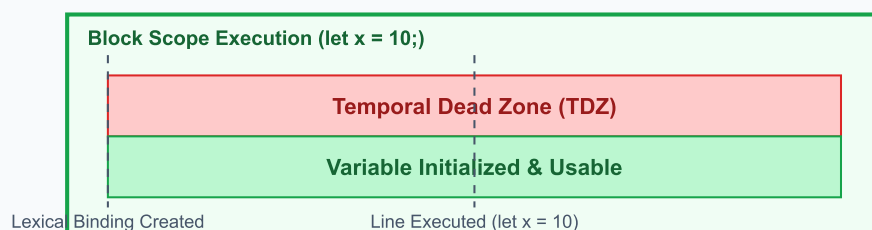


FIG 1. LIFECYCLE OF A LET/CONST LEXICAL BINDING

Part 2: Memory Models & Types

Is JavaScript pass-by-value or pass-by-reference?

It is strictly **pass-by-value**. For primitives, the actual value is copied. For objects, the "value" being copied and passed is a *reference* (a memory address pointer). Thus, mutating an object via a parameter mutates the original heap object, but reassigning the parameter only overwrites the local pointer copy.

What is the difference between primitive values and objects?

Primitives (String, Number, Boolean, Null, Undefined, Symbol, BigInt) are immutable, atomic data types stored directly on the stack (or registers). Objects (Arrays, Functions, Maps, plain objects) are mutable, complex structures stored in the Heap, interacted with purely via memory references.

What does `const` actually guarantee?

`const` only guarantees the immutability of the **binding** (the memory pointer). It guarantees that the variable cannot be reassigned to a new value or a new memory address. It does *not* guarantee the immutability of the underlying data structure itself (e.g., you can push to a `const` array).

Why does `typeof null` return "object" and `typeof []` return "object"?

`typeof [] === "object"` is correct by design; arrays are a specialized sub-type of objects. `typeof null === "object"` is a historical engine bug from 1995. Objects had a type tag of `000`, and `null` was represented as a machine-code NULL pointer (all zeros), accidentally matching the object tag.

What is the difference between `typeof` and `instanceof`?

`typeof` is a unary operator that returns a string representing the primitive type (with quirks for `null` and functions). `instanceof` is a binary operator that checks if an object's prototype chain contains the `.prototype` property of a specific constructor function.

Why is `Array.isArray()` better than `instanceof Array`?

`instanceof Array` fails across different JavaScript Realms (e.g., between an iframe and its parent). Each Realm has its own global `Array` constructor. `Array.isArray()` bypasses the prototype chain entirely, checking the engine's internal specification tags to verify if the object is genuinely an array.

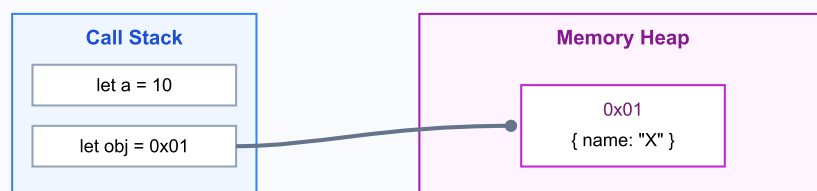


FIG 2. PRIMITIVES ON THE STACK VS OBJECT REFERENCES POINTING TO THE HEAP

Part 3: Object Semantics & Operators

What is the difference between object mutation and object reassignment?

Mutation alters the internal properties of an existing object in the heap (e.g., `obj.x = 10`) while the memory reference remains identical. Reassignment points the variable to an entirely new memory address in the heap (e.g., `obj = { x: 10 }`). `const` prevents reassignment, but permits mutation.

What is the difference between `==` and `===`?

`===` (Strict Equality) checks both value and type, returning false if types differ. `==` (Loose Equality) performs implicit type coercion if the types differ, utilizing abstract algorithms (like `ToNumber`) to force matching types before comparison.

What is the difference between `||` and `??`?

The logical OR (`||`) operator returns the right-hand operand if the left is *falsy* (evaluating `0`, `""`, `false`, `NaN`, `null`, `undefined` as false). The Nullish Coalescing (`??`) operator strictly returns the right-hand side *only* if the left is explicitly `null` or `undefined`, preserving values like `0` or `""`.

What is the difference between `?.` and normal property access?

Normal access (`obj.prop`) throws a fatal `TypeError` if `obj` is null or undefined. Optional Chaining (`?.`) safely short-circuits evaluation, returning `undefined` instead of crashing, making deep object traversal much safer.

What is the difference between `in` and checking `obj.prop !== undefined`?

The `in` operator checks if a key exists directly on the object *or anywhere in its prototype chain*, returning true even if the property's explicit value is set to `undefined`. Strict comparison (`!== undefined`) will return false if the property exists but holds the value `undefined`.

What is the difference between Map, Set, and ordinary objects?

Plain objects map strings/symbols to values, inherit from `Object.prototype`, and lack guaranteed iteration order. `Map` allows any data type (including objects) as keys, does not inherit prototype pollution vectors, and iterates in exact insertion order. `Set` stores strictly unique values of any type.

Why does `0.1 + 0.2` produce `0.30000000000000004`?

JavaScript uses the IEEE 754 standard for 64-bit floating-point math. Base-10 decimals like 0.1 and 0.2 cannot be precisely represented as binary fractions. The engine stores an approximation; adding them exposes the tiny binary rounding artifact.

Falsy Values	<code> </code> (Logical OR) Behavior	<code>??</code> (Nullish) Behavior
<code>null</code> , <code>undefined</code>	Returns right-hand side	Returns right-hand side
<code>0</code> , <code>""</code> , <code>false</code>	Returns right-hand side (Overrides valid falsy values)	Returns left-hand side (Preserves valid falsy values)

Part 4: V8 Engine Mechanics & Garbage Collection

How does JavaScript's garbage collector determine reachability?

Modern engines use a "Mark-and-Sweep" algorithm. They start at root references (global variables, active execution contexts) and traverse the entire object graph, "marking" everything they can reach. Afterward, they "sweep" unmarked objects in the heap, clearing their memory as they are mathematically unreachable by code.

What are the performance implications of using delete on properties?

Using `delete obj.prop` breaks the internal optimization structures (Hidden Classes/Shapes) in V8. The engine is forced to downgrade the object from a fast C++ struct representation into a slow "dictionary mode" (hash map), significantly degrading property access speeds.

How does object structure affect JavaScript engine optimization?

V8 creates "Hidden Classes" to track the memory layout of objects. Objects instantiated with identical properties in the exact same order share these Hidden Classes, enabling blazing-fast inline caching. Adding properties dynamically creates branch transitions; differing shapes defeat these optimizations.

How can careless use of dynamic object properties lead to prototype pollution?

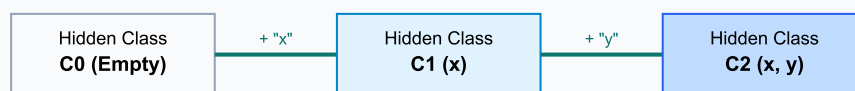
Because JavaScript relies on prototypical inheritance, recursive operations on unvalidated inputs (like parsing JSON) can allow an attacker to inject an `__proto__` key. This pollutes the global `Object.prototype`, altering the behavior of all objects in the runtime, opening up XSS or logic flaws.

How does the new operator work internally?

It executes four steps: 1) Creates a brand new empty object. 2) Links this new object's prototype (`[[Prototype]]`) to the constructor function's `.prototype` property. 3) Binds the `this` context to the new object and executes the constructor function. 4) Returns the object (unless the function explicitly returns another object).

What happens internally when using the + operator with different types?

The `+` operator handles both numeric addition and string concatenation. The engine runs the `ToPrimitive` algorithm. If either operand evaluates to a String (or converts to one), the engine coerces the other to a string and concatenates them. Otherwise, both are coerced to Numbers for math.



Adding properties dynamically forces transition chains.
Deleting properties fractures chains -> Dictionary Mode.

FIG 3. V8 HIDDEN CLASS TRANSITIONS (SHAPES) BASED ON PROPERTY ORDER

Part 5: Output-Based Scenarios (Logic & Operators)

Code Execution	Output	Explanation / Engine Mechanics
<pre>console.log(typeof null); console.log(typeof []);</pre>	<pre>"object" "object"</pre>	Legacy 32-bit bug for <code>null</code> . Arrays are technically specialized objects.
<pre>console.log(x); const x = 10;</pre>	<pre>ReferenceError or</pre>	<code>x</code> is hoisted but trapped in the Temporal Dead Zone until execution hits its declaration.
<pre>console.log("5" + 2); console.log("5" - 2);</pre>	<pre>"52" 3</pre>	<code>+</code> operator detects a string and concatenates. <code>-</code> operator strictly enforces numeric coercion via <code>ToNumber</code> .
<pre>console.log(0 "hi"); console.log(0 ?? "hi");</pre>	<pre>"hi" 0</pre>	<code> </code> yields the right side on any falsy value (0). <code>??</code> only yields right side if strictly null/undefined.
<pre>console.log([] == false); console.log([] === false);</pre>	<pre>true false</pre>	Loose <code>==</code> coerces both via <code>ToNumber</code> . <code>[]</code> becomes 0, <code>false</code> becomes 0. Strict <code>===</code> fails immediately on type difference.
<pre>console.log(true && "JS"); console.log(false "Dev");</pre>	<pre>"JS" "Dev"</pre>	Short-circuit evaluation. <code>&&</code> returns right side if left is truthy. <code> </code> returns right side if left is falsy.
<pre>const user = null; console.log(user?.name);</pre>	<pre>undefined</pre>	Optional chaining (<code>?.</code>) detects nullish left operand and safely aborts, returning undefined rather than crashing.

Part 6: Output-Based Scenarios (References & Memory)

Code Execution	Output	Explanation / Engine Mechanics
<pre>const a = {}; const b = {}; console.log(a === b);</pre>	false	Objects are compared by reference memory address. Each literal creates a distinct object in the heap.
<pre>const a = {}; const b = a; console.log(a === b);</pre>	true	Assigning an object copies the pointer. Both variables reference the exact same memory address.
<pre>const user = { age: 20 }; user.age = 21; console.log(user.ag e);</pre>	21	<code>const</code> prevents reassignment of the variable binding (the pointer). The heap object itself is fully mutable.
<pre>const x = 10; x = 20;</pre>	TypeError	Attempting to reassign a <code>const</code> binding to a new primitive value throws an error immediately.
<pre>let a = 10; let b = a; b = 20; console.log(a, b);</pre>	10, 20	Primitives are passed/assigned by value copy. Modifying <code>b</code> does not alter <code>a</code> .
<pre>const a = { x: 10 }; let b = a; b = { x: 20 }; console.log(a.x);</pre>	10	Reassigning <code>b</code> creates a completely new object in the heap and overwrites <code>b</code> 's pointer. The original object <code>a</code> points to is untouched.
<pre>function change(obj) { obj = { name: "Alex" }; } const user = { name: "Sam" }; change(user); console.log(user.na me);</pre>	"Sam"	JS passes references by value. Reassigning the parameter <code>obj</code> inside the function only changes the local variable's pointer, leaving the outer <code>user</code> untouched.